Michael Rios

COSC4336 21305 Fall 2025

u-sell-it.com Database

Executive Summary

For this project, I created a database for a website called u-sell-it.com, which is an online auction platform where sellers post items with a starting price, buyers place bids, and the highest bidder wins at the end. The main problem this database addresses is making it easier for people to buy and sell stuff online through auctions without everything getting chaotic or disorganized. It handles user accounts for both sellers and buyers, items listing with details like descriptions and prices, bids that track who's offering what, categories to organize things like electronics or cars, and even records of completed transactions. This way, the site can manage auctions smoothly, keep track of bids in real time, and ensure everything is secure and fair for users.

Design Documents

Analysis: Why do we use a design methodology (the database life cycle)?

When building a database, it's really important to follow a structured design methodology like the database life cycle because it helps avoid a bunch of headaches down the road. This approach breaks everything down into clear stages: starting with gathering requirements from users, then moving into conceptual design. Where you sketch out the big picture, logical design to map it to actual database structures, physical design for how it'll run on hardware, implementation to build it, and finally maintenance to keep it updated. By doing it this way, you can spot potential problems early, like if the data relationships don't make sense, make sure the database is efficient and scalable, and ensure it actually meets what people need. For my u-sell-it.com database project, this methodology helped me think through how bids connect to items and users without rushing into coding and regretting it later. It just makes the whole process more reliable and professional.

User requirements: What is the purpose of user requirements documentation?

The purpose of user requirements documentation is to lay out exactly what the end-users need and expect from the system, including any limitations or must-haves, so that the database design stays focused and useful. It acts as a roadmap to prevent building something that doesn't solve real problems. In this case, I represented the user requirements through the executive summary above.

Conceptual Design:

What is the definition of conceptual design?

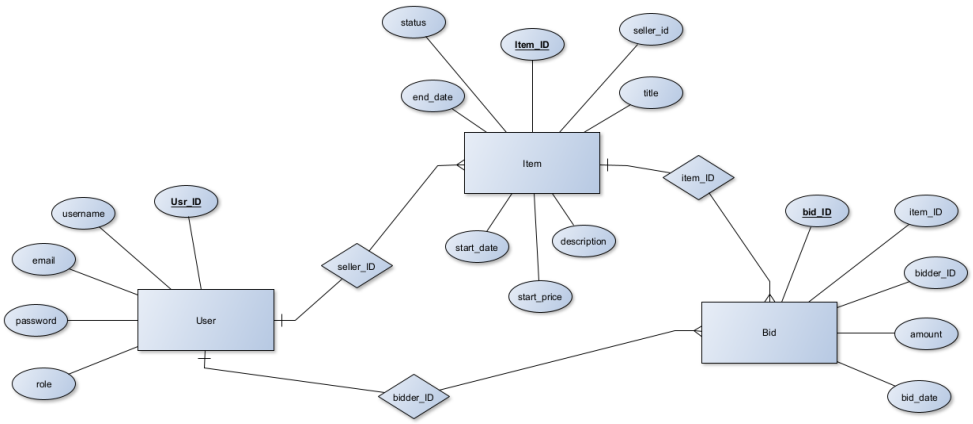
Conceptual design is basically the first big step in planning a database, where you focus on the high level stuff like identifying the main entities (like users or items), their key attributes, and how they relate to each other, all without getting bogged down in technical details like what software or data types to use.

What is the purpose of this design phase?

The goal here is to build a simple, clear model that's independent of any specific technology, so it captures the core ideas and relationships in a way that's easy to understand and refine before diving into the nitty-gritty. It helps ensure the database will make sense logically and support the application's needs right from the start.

We use an ERD to represent the conceptual design phase, what is an ERD?

An ERD, or Entity-Relationship Diagram, is a visual tool that shows the entities in your system as boxes, lists their attributes inside, and uses lines with symbols to illustrate the relationships between them, like one-to-one or many-to-many, which makes it easier to see the overall structure.



For u-sell-it.com auction database, the ERD includes these main entities and how they connect:

- The **User** entity has attributes like user_ID (primary key), username, email, password, and role (to distinguish sellers from buyers).

- The **Item** entity covers item_ID (primary key), title, description, start_price, end_date, status (like “active” or “sold”), and seller_ID (foreign key to User).
- The **Bid** entity has bid_ID (primary key) item_ID (foreign key to Item), bidder_ID (foreign key to User), amount, and bid_date.
- As for relationships: A single User can list multiple Items as sellers (one-to-many); an Item can receive multiple Bids (one-to-many), and a User can list multiple Bids as a bidder (one-to-many).

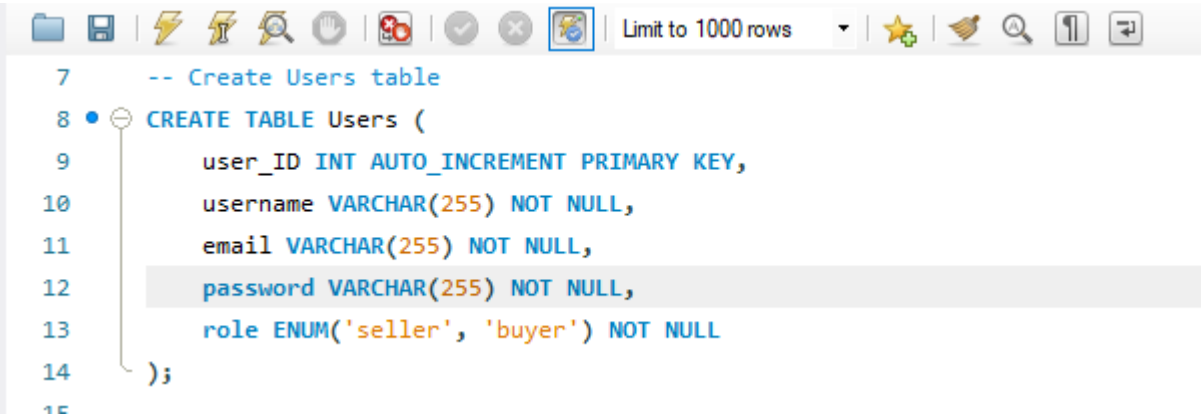
Database Build

I created the database based on a data dictionary I put together, which outlines all the tables, fields, data types, and keys. This kept everything organized and consistent.

The data dictionary looks like this:

- Users: user_ID (INTEGER, primay key), username (TEXT), email (TEXT), password (TEXT), role (TEXT).
- Items: item_ID (INTEGER, primary key), title (TEXT), description (TEXT), start_price (DECIMAL), end_date (DATE), status (TEXT), seller_ID (INTEGER, foreign key to User).
- Bids: bid_ID (INTEGER, primary key), item_ID (INTEGER, foreign key to Items), bidder_ID (INTEGER, foreign key to Users), amount (DECIMAL), bid_date (DATE).

SQL code:



I added some constraints like NOT NULL and UNIQUE to make sure the data stays clean, for example, no duplicate emails and required fields for security.

Enter Data

I entered some made up data to test things out, making sure there were at least 5 records in the main User table, which acts as the demographic hub since everything revolves around users. For the other tables, I added enough to simulate real auctions without overdoing it.

In the User table:

```
4   -- Insert into Users
5 • INSERT INTO Users (username, email, password, role) VALUES
6   ('mrios', 'mrios@example.com', 'securepass1', 'seller'),
7   ('jane', 'jane@example.com', 'securepass2', 'buyer'),
8   ('dave', 'dave@example.com', 'securepass3', 'seller'),
9   ('alex', 'alex@example.com', 'securepass4', 'buyer'),
10  ('eve', 'eve@example.com', 'securepass5', 'buyer');
```

For Items, I added 6 listings, like a vintage watch with item_ID 1, title ‘Vintage Watch’, description ‘Classic timepiece’, start_price 100.00, end_date 2025-11-15, status ‘active’, seller_ID 1. Bids got about 10 entries with different amounts to simulate bidding wars.

Critical Analysis

When I built the final database, it mostly stuck to my initial design, but I did make a couple of tweaks along the way, which was fine since the point is to learn from it. For example, as I started entering data, I realized the bid_date field should include time, not just date, becuase auctions could have bids coming in at the end, so I changed it to DATETIME instead of DATE. Also, I added a CHECK constraint to the role field in Users to limit it to ‘seller’ or ‘buyer’, which I hadn’t thought of at first, but it helped prevent bad data. In the Items table, the description field ended up needing to be longer than I planned because some item details were more detailed, like including condition or shopping info. Overall, these changes made the database better, and next time I’ll remember to think more about real-world data variations upfront. It was a good lesson in flexibility.

Application (User interface)

For the user interface, I designed a basic app to help users work with the database. Even though it doesn't need to be functional yet, it's more about imagining how it would look and operate listing items or bidding.

Forms

I designed two forms: one for the main table and one for a related table, with the possibility to embed or link them.

- **Main Form: User Profile** This form uses data from the Users table, showing username, email, role, and maybe a spot to update the password. It has navigation buttons to move between users or edit info, and I embedded a subforum, showing Items for sellers or Bids for buyers based on their role.
- **Subforum: Item Bids** (embedded in an Item Details form, linked from User Profile) This pulls from the Bids table for a specific item, displaying bid_ID, bidder username (linked from Users), amount, and bid_date, filtered by item_ID to list all bids on that auction for easy tracking of the top bid.

These forms are just a blueprint for now, but they're set up to be user-friendly and logical.

Analysis of form design

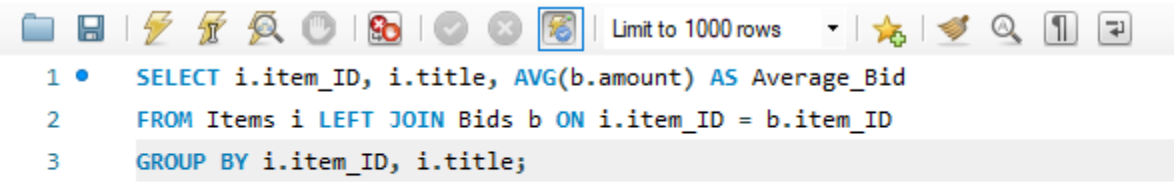
- My design keeps things consistent with uniform button styles and layouts, so users won't feel lost. I grouped related fields, user clear labels, and locked IDs to avoid errors. The embedded subforums cut down on navigation, saving time and fitting usability goals like reducing effort and giving clear feedback.
- For older adults, I'd bump up font sizes and use high-contrast colors to help with seeing, make buttons bigger for easier taps, simplify the layout by trimming extra options or adding tooltips, and maybe add voice commands or screen reader support for and physical or tech challenges.

Queries

I wrote four queries to show off different SQL skills, covering joins, aggregates, grouping, and sorting, each with a name, user-friendly purpose, the SQL code, and sample results for reference.

Query 1

- Query name: AvdBidPerItem
- User Purpose: As a seller, I want to check the average price bid on my items to see if my starting price was reasonable or if there's good interest.
- SQL statement



- Results:

item_ID	title	Average_Bid
1	Vintage Watch	123.750000
2	Used Bicycle	NULL
3	Laptop Computer	405.000000
4	Antique Book	110.000000
5	Smartphone	300.000000
6	Gaming Console	110.000000

(This uses a join, aggregate (AVG), and GROUP BY.)

Query 2

- Query Name: BidsSortedByDate
- User Purpose: To look at recent bidding activity, see the latest bids with item and bidder details, sort newest first.
- SQL statement:

```sql
SELECT b.bid_ID, i.title AS item, u.username AS bidder, b.amount, b.bid_date
FROM Bids b
JOIN Items i ON b.item_ID = i.item_ID
JOIN Users u ON b.bidder_ID = u.user_ID
ORDER BY b.bid_date DESC;
```

- Results:

bid_ID	item	bidder	amount	bid_date
8	Smartphone	alex	300.00	2025-11-14 13:22:00
6	Antique Book	alex	110.00	2025-11-10 10:45:00
7	Antique Book	eve	110.00	2025-11-08 10:00:00
4	Laptop Computer	eve	400.00	2025-11-05 12:00:00
10	Vintage Watch	dave	135.00	2025-11-05 01:13:00
5	Laptop Computer	jane	410.00	2025-11-04 13:00:00
9	Vintage Watch	jane	130.00	2025-11-04 06:49:00
3	Gaming Console	jane	110.00	2025-11-03 15:00:00
2	Vintage Watch	alex	120.00	2025-11-02 10:35:00
1	Vintage Watch	eve	110.00	2025-11-02 10:00:00

(This joins three tables and sorts.)

Query 3

- Query Name: BidCountPerUser
- User Purpose: For admins or users to see how many bids each buyer has made, maybe to find active users or review personal activity.
- SQL statement

```sql
SELECT u.user_ID, u.username, COUNT(b.bid_ID) AS Num_bids
FROM Users u LEFT JOIN Bids b ON u.user_ID = b.bidder_ID
WHERE u.role = 'buyer'
GROUP BY u.user_ID, u.username;
```

- Results:

	user_ID	username	Num_bids
▶	2	jane	3
	4	alex	3
	5	eve	3

(This join, uses COUNT aggregate, and GROUP BY with a buyer filter.)

Query 4

- Query Name: HighBidItems
- User Purpose: To spot items with bids above the site's average, sorted by highest bids, so sellers can see hot auctions and buyers can find good deals.
- SQL statement


```
1  SELECT i.item_ID, i.title, MAX(b.amount) AS Highest_Bid
2  FROM Items i JOIN Bids b ON i.item_ID = b.item_ID
3  GROUP BY i.item_ID, i.title
4  HAVING MAX(b.amount) > (SELECT AVG(amount) FROM bids)
5  ORDER BY Highest_Bid DESC;
```

- Results:

	item_ID	title	Highest_Bid
▶	3	Laptop Computer	410.00
	5	Smartphone	300.00

(This joins, uses MAX and subquery AVG, GROUP BY, HAVING, and sorts.)